Documentación del código: tarea_compasion_final_v3.py

Explicación línea por línea

BLOQUE 1: Codificación y compatibilidad (líneas 1-6)

```
# -*- coding: utf-8 -*-
```

Declara que el archivo usa codificación UTF-8. Esto permite usar caracteres con tilde (á, é, ñ) en el código fuente sin errores de lectura.

```
import sys, io

if hasattr(sys.stdout, 'buffer'):

    sys.stdout = io.TextIOWrapper(sys.stdout.buffer, encoding='ascii', errors='replace')

    sys.stderr = io.TextIOWrapper(sys.stderr.buffer, encoding='ascii', errors='replace')
```

¿Por qué se necesita esto? Cuando ejecutas el script desde el IDE de PsychoPy en Windows, PsychoPy crea un "tubo" (pipe) para leer lo que el script imprime en consola. Este tubo usa la codificación del sistema (cp1252 en Windows en español), que no soporta todos los caracteres UTF-8. Si un print() contiene un carácter que cp1252 no reconoce, PsychoPy crashea con un error de codecs.py.

La solución: reconfigurar sys.stdout y sys.stderr para que conviertan todo a ASCII puro, reemplazando caracteres problemáticos con ?. Esto solo afecta la consola del investigador, no lo que ve el participante en pantalla.

- sys — Módulo estándar de Python para acceder a configuración del sistema (stdout, stderr, argumentos).
- io — Módulo estándar para manejo de flujos de entrada/salida. TextIOWrapper envuelve un flujo binario y le aplica una codificación de texto.

BLOQUE 2: Configuración de audio de PsychoPy (líneas 8-12)

from psychopy import prefs

prefs.hardware['audioLib'] = ['PTB']

prefs.hardware['audioLatencyMode'] = 3

¿Por qué va antes de las otras importaciones? PsychoPy configura su sistema de audio en el momento en que importas visual o core. Si no estableces las preferencias antes, usa la configuración por defecto que tiene mayor latencia.

- PTB (Psychtoolbox) — Es el backend de audio con menor latencia disponible en PsychoPy. Importante porque los videos del experimento tienen audio.
- audioLatencyMode = 3 — Modo de latencia más baja. PsychoPy tiene 4 modos (0-3); el modo 3 prioriza la precisión temporal sobre la estabilidad.

BLOQUE 3: Importaciones principales (líneas 14-28)

import pandas as pd

Librería para manipulación de datos tabulares. Se usa para leer el Excel de condiciones (condiciones_a.xlsx / condiciones_b.xlsx) y para guardar los datos de salida en CSV.

from psychopy import visual, core, event, gui, logging, monitors

Módulos de PsychoPy. Cada uno cumple un rol específico:

- visual — Crear y dibujar estímulos en pantalla (texto, videos, sliders). También contiene Window (la ventana del experimento) y MovieStim (reproductor de video).
- core — Funciones de tiempo: core.wait() para pausas, core.getTime() para timestamps, core.quit() para cerrar el programa.
- event — Captura de teclado: event.getKeys() para leer teclas presionadas, event.waitKeys() para esperar una tecla específica.
- gui — Diálogos gráficos. Se usa gui.DlgFromDict() para la ventana inicial donde se ingresa participante, sesión y grupo.
- logging — Sistema de registro de PsychoPy. Se configura para mostrar solo warnings y errores, no mensajes informativos (que enlentecen la ejecución).
- monitors — Permite definir las características físicas del monitor (tamaño, distancia, resolución) para que PsychoPy calcule correctamente las unidades visuales.

import os

Operaciones del sistema de archivos: cambiar directorio (os.chdir), construir rutas (os.path.join), verificar si archivos existen (os.path.exists), crear carpetas (os.makedirs).

import gc

Garbage collector (recolector de basura). Se usa para liberar memoria explícitamente después de cerrar un video. Los videos consumen mucha RAM y si no se liberan manualmente, pueden acumularse y causar problemas.

import serial

import serial.tools.list_ports

Comunicación serial con el Arduino. serial.Serial abre el puerto COM; serial.tools.list_ports lista todos los puertos disponibles para autodetectar en cuál está conectado el Arduino.

from datetime import datetime

Para generar el timestamp del nombre del archivo de salida (día, mes, año, hora, minuto, segundo).

```
try:

    from pylsl import StreamInfo, StreamOutlet

    LSL_DISPONIBLE = True

except ImportError:

    print("AVISO: pylsl no instalado. Marcadores LSL desactivados.")

    LSL_DISPONIBLE = False
```

pylsl (Python Lab Streaming Layer) — Protocolo estándar para enviar marcadores de eventos a equipos de neurofisiología como el ANT Neuro. Los marcadores viajan por la red local (TCP/IP) y se sincronizan con la señal EEG.

El try/except es un mecanismo de seguridad: si pylsl no está instalado, el experimento funciona igual pero sin enviar marcadores EEG. Esto permite correr el script en cualquier computador para pruebas sin necesitar el equipo EEG conectado.

- StreamInfo — Define las características del stream (nombre, tipo, canales).
- StreamOutlet — El "grifo" por donde salen los marcadores.

BLOQUE 4: Constantes de configuración (líneas 30-60)

RUTA_FIJA = r'C:\Users\p_ull\OneDrive\Documentos\all_data_tarea_experimento\experimento_UMCE'

Ruta raíz del experimento. La r antes de las comillas indica "raw string" — le dice a Python que no interprete los backslash \ como caracteres especiales (como \n para salto de línea).

NOMBRE_CARPETA_VIDEOS = 'stimuli_video_mp4'

CARPETA_SALIDA = os.path.join(RUTA_FIJA, 'datos_salida')

Subcarpetas dentro de la ruta raíz. os.path.join construye la ruta completa de forma segura independientemente del sistema operativo.

TIEMPO_FIJACION = 5.0 # Cruz de fijación: 5 segundos

TIEMPO_CONTEXTO = 10.0 # Texto de contexto: 10 segundos

TIEMPO_VIDEO = 10.0 # Video: 10 segundos (pantalla negra si termina antes)

TIEMPO_RATING = 6.0 # Valoración: 6 segundos

Tiempos de cada fase del trial en segundos. Están como constantes al inicio para poder modificarlos fácilmente sin tocar el resto del código.

VIDEO_SIZE = (960, 720)

Resolución de renderizado del video en píxeles (ancho x alto). Los videos originales son 320x240 (ratio 4:3) y se escalan a 960x720 para verse bien en un monitor de 1920x1200.

TRIALS_ENTRENAMIENTO = 2

Cantidad fija de trials de entrenamiento. Si el Excel tiene más filas de entrenamiento, se seleccionan 2 al azar con videos únicos.

LSL_FIJACION = 1

LSL_CONTEXTO = 2

LSL_VIDEO = 3

LSL_RATING = 4

Valores numéricos de los marcadores LSL. Cada número identifica una fase del trial en el registro EEG. Cuando analices el EEG, puedes segmentar la señal buscando estos marcadores.

ARDUINO_BAUDRATE = 115200

ARDUINO_PORT = None

Configuración serial del Arduino. 115200 baudios es la velocidad de comunicación (debe coincidir con el sketch del Arduino). None activa la autodetección del puerto COM.

logging.console.setLevel(logging.WARNING)

Suprime los mensajes informativos de PsychoPy en consola. Solo muestra warnings y errores. Los mensajes de debug generan overhead innecesario durante el experimento.

BLOQUE 5: Inicialización del entorno (líneas 62-89)

os.chdir(RUTA_FIJA)

Cambia el directorio de trabajo a la carpeta del experimento. Esto permite usar rutas relativas (como condiciones_a.xlsx) en vez de rutas absolutas.

os.makedirs(CARPETA_SALIDA, exist_ok=True)

Crea la carpeta datos_salida si no existe. exist_ok=True evita un error si ya existe.

info = {'Participante': '001', 'Sesion': '1', 'Grupo': ['A', 'B']}

if not gui.DlgFromDict(dictionary=info, title='Tarea Compasión').OK:

    core.quit()

Muestra la ventana de diálogo inicial. Grupo se define como lista ['A', 'B'] para que PsychoPy lo muestre como un menú desplegable (dropdown) en vez de un campo de texto libre. Si el usuario presiona Cancelar, el script se cierra.

```
grupo = info['Grupo'].upper().strip()
```

Normaliza la selección: convierte a mayúscula y elimina espacios. Así 'a', ' A ' y 'A' se tratan igual.

```
timestamp_inicio = datetime.now().strftime('%d%m%Y%H%M%S')

nombre_archivo_salida = os.path.join(

    CARPETA_SALIDA,

    f"{info['Participante']}_{grupo}_{timestamp_inicio}.csv"

)
```

Genera el nombre del archivo de salida. Ejemplo: 001_A_18042026143015.csv (ID_Grupo_díamesañohoraminutosegundo).

```
archivo_condiciones = f'condiciones_{grupo.lower()}.xlsx'
```

Selecciona el archivo de condiciones según el grupo: condiciones_a.xlsx o condiciones_b.xlsx.

BLOQUE 6: Triggers BNC para el Trigno (líneas 91-98)

```
TRIG_START = ord('H')  # 72

TRIG_STOP = ord('L')   # 76
```

Códigos ASCII que se envían al Arduino. El Arduino está programado para:

- Recibir 'H' (72) → activar pin 2 → pulso en BNC 1 → Trigno inicia grabación EMG.
- Recibir 'L' (76) → activar pin 3 → pulso en BNC 2 → Trigno detiene grabación EMG.

ord() convierte el carácter a su valor numérico ASCII.

BLOQUE 7: Funciones de Arduino (líneas 100-172)

detectar_arduino() (líneas 103-112)

Recorre todos los puertos COM del sistema y busca uno cuya descripción o fabricante contenga palabras clave como "arduino", "ch340" (chip USB común en Arduinos clónicos), o "usb-serial". Retorna el nombre del puerto (ej: 'COM3') o None si no encuentra nada.

conectar_arduino() (líneas 115-155)

Abre la conexión serial con el Arduino:

1. Reset DTR (líneas 133-136): Baja y sube la señal DTR, lo que reinicia el Arduino. Esto asegura que el Arduino esté en un estado conocido. Se esperan 2 segundos para que el bootloader termine.

2. Flush buffers (líneas 137-138): Limpia los buffers de entrada y salida. Cualquier dato residual de conexiones anteriores se descarta.

3. Esperar READY (líneas 140-148): El sketch del Arduino imprime "READY" cuando termina su setup(). El script espera hasta 15 intentos (1.5 segundos) para recibir esta confirmación.

4. Fallback (línea 150): Si no recibe READY pero el puerto se abrió sin error, asume que el Arduino está funcionando.

arduino = conectar_arduino()

MODO_SIMULACION = (arduino is None)

Si no hay Arduino conectado, el experimento corre en modo simulación: todas las funciones de trigger retornan timestamps pero no envían nada por serial.

enviar_trigger(codigo) (líneas 162-172)

Envía un byte al Arduino y retorna el timestamp exacto del envío. La latencia es ~1.1ms (constante): 0.1ms del serial.write + 1ms del USB. En modo simulación, solo retorna el timestamp.

BLOQUE 8: Stream LSL para ANT Neuro (líneas 175-190)

lsl_info = StreamInfo('CompasionTask', 'Markers', 1, 0, 'int32', 'compasion_markers')

```
lsl_outlet = StreamOutlet(lsl_info)
```

Crea un stream LSL con estas propiedades:

- 'CompasionTask' — Nombre del stream (visible para el ANT Neuro).
- 'Markers' — Tipo de contenido. Los programas que reciben marcadores buscan streams de tipo "Markers".
- 1 — Un solo canal (enviamos un número por marcador).
- 0 — Tasa de muestreo irregular (los marcadores no se envían a intervalos fijos).
- 'int32' — Formato de datos: entero de 32 bits.
- 'compasion_markers' — Identificador único del stream.

enviar_lsl(marker) (líneas 187-190)

Envía un marcador entero al stream LSL. push_sample([marker]) empuja una muestra de un canal con el valor del marcador. Si el stream no se creó (porque pylsl no está instalado), la función no hace nada.

BLOQUE 9: Ventana y monitor (líneas 193-229)

```
mon = monitors.Monitor('expMonitor')

mon.setWidth(52)        # Ancho del monitor en cm

mon.setDistance(60)       # Distancia del participante al monitor en cm

mon.setSizePix([1920, 1200])  # Resolución nativa

mon.save()
```

Define un perfil de monitor. PsychoPy usa estos datos para convertir entre unidades (cm, grados visuales, píxeles). Sin este perfil, PsychoPy genera un warning que puede afectar el timing.

```
win = visual.Window(

    size=[1920, 1200],

    monitor='expMonitor',

    fullscr=True,
```

```
    color=[0, 0, 0],      # Negro

    units='norm',         # Unidades normalizadas (-1 a 1)

    waitBlanking=True,    # Sincronizar con vsync del monitor

    allowGUI=False

)

win.mouseVisible = False
```

Crea la ventana del experimento en pantalla completa:

- fullscr=True — Pantalla completa, esencial para vsync confiable.
- waitBlanking=True — win.flip() espera al refresco vertical del monitor antes de retornar. Esto garantiza que cada frame aparece en un momento predecible.
- units='norm' — Coordenadas normalizadas: (0,0) es el centro, (-1,-1) la esquina inferior izquierda, (1,1) la superior derecha.
- allowGUI=False — Oculta la barra de título.

```
fps_real = win.getActualFrameRate(nIdentical=60, nMaxFrames=120, nWarmUpFrames=20)
```

Mide la tasa de refresco real del monitor (ej: 59.94 Hz en vez del nominal de 60 Hz). Esto es crítico porque la duración de cada frame se calcula a partir de este valor.

```
FRAMES_FIJACION = int(round(TIEMPO_FIJACION / FRAME_DUR))
```

Convierte los tiempos en segundos a número de frames. Ejemplo: 5.0s / 0.01667s = 300 frames. ¿Por qué contar frames en vez de usar core.wait()? Porque core.wait() tiene jitter de ±1-5ms. Contar frames tiene jitter de ±1 frame (~16.7ms a 60Hz), pero es determinístico y constante entre trials, lo cual es mejor para análisis fisiológico.

BLOQUE 10: Carga de condiciones y balanceo (líneas 231-313)

```
df = pd.read_excel(archivo_condiciones)
```

Lee el Excel seleccionado (condiciones_a.xlsx o condiciones_b.xlsx).

Normalización de columnas (líneas 241-258)

```
col_map = {}

for col in df.columns:

    col_limpio = col.strip().lower()

    if col_limpio in ('condición', 'condicion'):

        col_map[col] = 'condicion'

    elif col_limpio in ('nombre_video', 'video'):

        col_map[col] = 'nombre_video'

    ...

df = df.rename(columns=col_map)
```

Mapea las columnas del Excel a nombres internos estandarizados. Esto hace el código robusto a variaciones: si el Excel dice Video, video, Condición o condicion, todo funciona igual. Después verifica que las columnas requeridas existan.

Separación entrenamiento/experimental (líneas 260-278)

```
df_entrenamiento = df[df['tipo'] == 'entrenamiento'].copy()

df_experimental = df[df['tipo'] != 'entrenamiento'].copy()
```

Filtra las filas según la columna tipo. .copy() crea una copia independiente para evitar warnings de pandas al modificar un slice del DataFrame original.

```
df_entrenamiento = df_entrenamiento.drop_duplicates(subset=['nombre_video'])

if len(df_entrenamiento) > TRIALS_ENTRENAMIENTO:

    df_entrenamiento = df_entrenamiento.sample(n=TRIALS_ENTRENAMIENTO)
```

Garantiza que no se repita el mismo video en entrenamiento. Si hay más de 2 opciones, selecciona 2 al azar.

Balanceo experimental (líneas 280-306)

```
for cond in condiciones_unicas:

    df_cond = df_experimental[df_experimental['condicion'].str.strip() == cond].copy()

    df_cond = df_cond.drop_duplicates(subset=['nombre_video'])

    seleccion = df_cond.sample(n=TRIALS_POR_CONDICION)

    seleccion = seleccion.sample(frac=1).reset_index(drop=True)  # Aleatorizar dentro

    trials_balanceados.append(seleccion)
```

Para cada condición (responsable, noresponsable, neutro):

1. Filtra las filas de esa condición.
2. Elimina videos duplicados.
3. Selecciona exactamente 10 videos al azar.
4. Aleatoriza el orden dentro de esa condición.

```
df_final = pd.concat(trials_balanceados, ignore_index=True)

df_final = df_final.sample(frac=1).reset_index(drop=True)
```

Combina las 3 condiciones y mezcla el orden total. Resultado: 30 trials experimentales con 10 por condición, orden completamente aleatorio.

BLOQUE 11: Estímulos visuales reutilizables (líneas 315-334)

```
cruz = visual.TextStim(win, text='+', color='white', height=0.1)

texto_contexto = visual.TextStim(win, text='', color='white', height=0.07, wrapWidth=1.5)

texto_error = visual.TextStim(win, text='Error: Video no encontrado', color='red', height=0.06)
```

Se crean una sola vez y se reutilizan en todos los trials. Crear objetos visuales tiene un costo computacional; hacerlo dentro del bucle de trials introduciría latencia variable.

```
slider_visual = visual.Slider(
```

```
    win, ticks=(1, 2, 3, 4, 5, 6, 7, 8, 9),

    labels=['1\nMuy negativo', '2', '3', '4', '5\nNeutral', '6', '7', '8', '9\nMuy positivo'],

    granularity=1, style='slider', size=(1.0, 0.05),

    pos=(0, -0.1), color='white', labelHeight=0.045, font='Arial'

)
```

Slider con escala 1-9, granularidad de 1 (solo valores enteros). Los ticks de anclaje (1, 5, 9) tienen etiquetas descriptivas; los intermedios solo muestran el número.

BLOQUE 12: Funciones de video y cierre (líneas 337-389)

precargar_video(video_path) (líneas 340-358)

Carga el video en RAM antes de necesitarlo. Esto se ejecuta durante la fase de fijación (mientras el participante ve la cruz), eliminando la latencia de carga entre el contexto y el video.

Intenta primero con audio (noAudio=False). Si falla (por ejemplo, por un codec faltante), intenta sin audio como fallback. VIDEO_SIZE es necesario porque PsychoPy 2025 requiere tamaño explícito en MovieStim.

limpiar_video(mov) (líneas 361-372)

Detiene la reproducción, libera el objeto, y fuerza la recolección de basura. Sin esto, los recursos del video (memoria de video, handles de archivo, buffers de audio) se acumularían trial tras trial.

cerrar_todo(datos, archivo) (líneas 375-389)

Función de cierre de emergencia. Si el experimentador presiona Escape en cualquier momento:

1. Guarda los datos recopilados hasta ese punto con prefijo PARCIAL_.
2. Cierra la conexión Arduino.
3. Cierra la ventana.
4. Termina el programa.

BLOQUE 13: Función principal del trial (líneas 392-514)

ejecutar_trial(trial, num_trial, fase)

Esta es la función central del experimento. Ejecuta las 4 fases de un trial y retorna un diccionario con todos los datos registrados.

FASE 1: Fijación + Precarga (líneas 401-414)

```
enviar_lsl(LSL_FIJACION)    # Marcador EEG: inicio fijación

cruz.draw()

t_fijacion = win.flip()      # Timestamp exacto del momento en que la cruz aparece

mov = precargar_video(...)    # Cargar video mientras el participante ve la cruz
```

El video se carga durante los 5 segundos de fijación (zona "no crítica" temporalmente). Así cuando llega la fase de video, ya está listo en RAM.

FASE 2: Contexto (líneas 416-427)

```
enviar_lsl(LSL_CONTEXTO)

texto_contexto.text = trial['contexto']

texto_contexto.draw()

t_contexto = win.flip()
```

Muestra el texto de contexto durante 10 segundos. El marcador LSL se envía justo antes del flip para que coincida lo más posible con el momento en que el estímulo aparece en pantalla.

FASE 3: Video (líneas 429-469)

```
enviar_lsl(LSL_VIDEO)

mov.draw()

t_video = win.flip()
```

El video se reproduce frame a frame dentro de un bucle de FRAMES_VIDEO frames (10 segundos). Si el video termina antes de los 10 segundos:

- Se limpia el video de memoria.
- Se sigue haciendo win.flip() sin dibujar nada → pantalla negra.
- Se garantizan exactamente 10 segundos independientemente de la duración del video.

La detección de fin del video es compatible con dos APIs de PsychoPy (isFinished y status == FINISHED) porque diferentes versiones usan diferentes mecanismos.

FASE 4: Rating (líneas 471-494)

enviar_lsl(LSL_RATING)

valor = 5 # Inicia en Neutral

El slider comienza en 5 (Neutral). El participante tiene 6 segundos para mover el marcador con las flechas izquierda/derecha. Cada pulsación mueve el valor en 1 unidad, con límites en 1 y 9.

Registro de datos (líneas 496-514)

Retorna un diccionario con todos los datos del trial:

- Identificación: Participante, Sesión, Grupo, Trial, Fase.
- Estímulo: Video, Condición.
- Respuesta: Rating (1-9).
- Calidad: Audio_Video_OK.
- Timestamps: T_Fijacion, T_Contexto, T_Video_Inicio, T_Video_Fin, T_Rating, T_Start_Grabacion.
- Técnico: Frame_Hz.

Todos los timestamps tienen precisión de 6 decimales (microsegundos). Estos permiten segmentar la grabación continua del EMG en el análisis posterior.

BLOQUE 14: Flujo principal del experimento (líneas 517-661)

Estabilización del Arduino (líneas 520-524)

if not MODO_SIMULACION:

```
core.wait(3.0)

if arduino:

    arduino.flushInput()
```

Espera 3 segundos después del reset DTR para que el Arduino se estabilice completamente. Sin esto, los primeros triggers podrían perderse.

Pantalla de configuración BNC (líneas 526-540)

Pantalla para el investigador (no el participante). Permite verificar la conexión BNC con el Trigno antes de iniciar el experimento.

Instrucciones al participante (líneas 542-567)

Dos pantallas de instrucciones controladas por barra espaciadora. La instrucción 1 explica la estructura general; la instrucción 2 anuncia el entrenamiento y explica cómo responder.

Inicio de grabación EMG (líneas 569-576)

```
t_start_grabacion = enviar_trigger(TRIG_START)

core.wait(0.5)
```

Envía el trigger de inicio al Trigno. La espera de 0.5 segundos permite que el Trigno procese el trigger e inicie la grabación antes de que comiencen los trials.

Bucle de entrenamiento (líneas 578-592)

Ejecuta los 2 trials de entrenamiento con `fase='entrenamiento'`. Los datos se guardan igual que los experimentales pero se pueden filtrar por la columna `Fase` en el análisis.

Instrucción 3: transición a experimental (líneas 594-602)

Avisa al participante que el entrenamiento terminó y que presione espacio para iniciar el experimento real.

Bucle experimental (líneas 604-615)

Ejecuta los 30 trials experimentales. Después de cada trial, limpia el buffer serial del Arduino para evitar acumulación de datos residuales.

Stop grabación y guardado (líneas 617-643)

```
t_stop_grabacion = enviar_trigger(TRIG_STOP)
```

```
df_salida = pd.DataFrame(datos_guardados)

df_salida['T_Stop_Grabacion'] = round(t_stop_grabacion, 6)

df_salida.to_csv(nombre_archivo_salida, index=False)
```

1. Envía trigger de stop al Trigno → detiene grabación EMG.
2. Convierte la lista de diccionarios a un DataFrame de pandas.
3. Agrega el timestamp de stop a todas las filas.
4. Guarda como CSV.

El reporte final en consola muestra un resumen: cantidad de trials, errores de audio, y ruta del archivo guardado.

Pantalla final y cierre limpio (líneas 645-661)

```
visual.TextStim(win, text='Gracias por participar...').draw()

win.flip()

core.wait(3)

arduino.close()

win.close()

core.quit()
```

Muestra el agradecimiento 3 segundos, cierra el puerto serial del Arduino, cierra la ventana de PsychoPy, y termina el proceso.

Resumen de los dos sistemas de triggers

Sistema	Dispositivo	Conexión	Función	Cuándo se usa
BNC (Arduino)	Trigno Discover (EMG)	USB Serial → BNC	Start/Stop de grabación continua	1 vez al inicio, 1 vez al final

Sistema	Dispositivo	Conexión	Función	Cuándo se usa
LSL (pylsl)	ANT Neuro (EEG)	Red TCP/IP	Marcadores por fase de cada trial	4 veces por trial (fijación, contexto, video, rating)

Estructura del CSV de salida

Columna	Descripción
Participante	ID del participante
Sesion	Número de sesión
Grupo	A o B (set de estímulos)
Trial	Número de trial global (1-32)
Fase	"entrenamiento" o "experimental"
Video	Nombre del archivo de video
Condicion	responsable / noresponsable / neutro
Rating	Valoración emocional (1-9)
Audio_Video_OK	Si el video reprodujo audio correctamente
T_Fijacion	Timestamp inicio cruz de fijación
T_Contexto	Timestamp inicio texto de contexto
T_Video_Inicio	Timestamp primer frame del video
T_Video_Fin	Timestamp fin de la fase de video
T_Rating	Timestamp inicio pantalla de rating
T_Start_Grabacion	Timestamp del trigger START (EMG)
T_Stop_Grabacion	Timestamp del trigger STOP (EMG)
Frame_Hz	Tasa de refresco real del monitor